

Design Patterns

Quick Reference

Plain-language explanations of each pattern, why it was used in this codebase, and the points to keep in mind while designing it yourself.

01	Abstract Factory	Creational
02	Builder	Creational
03	Factory Method	Creational
04	Singleton	Creational
05	Adapter	Structural
06	Decorator	Structural
07	Facade	Structural
08	Proxy	Structural
09	Chain of Responsibility	Behavioral
10	Observer	Behavioral
11	Strategy	Behavioral

Based on: CoreDesignPattern package — AbstractFactory, Builder, Factory, Singleton, Adapter, Decorator, Facade, Proxy, Chain_of_Responsibility, Observer, Strategy

Abstract Factory

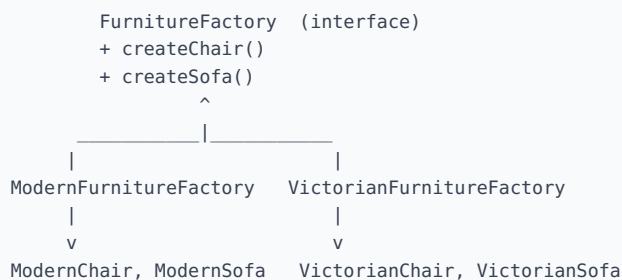
CoreDesignPattern.AbstractFactory

What is it?

Abstract Factory gives you one factory whose job is to create a whole **family of related objects** together. Instead of calling `new` all over your code, you ask the factory for what you need, and it hands you objects that are guaranteed to belong together.

Why it was used here

In the furniture example, `FurnitureFactory` can create a `Chair` and a `Sofa`. Pick `ModernFurnitureFactory` and you get a matching Modern Chair + Modern Sofa. Switch to `VictorianFurnitureFactory` and you get a matching Victorian pair instead. The client never has to know the concrete classes — it just can't accidentally mix a Modern Chair with a Victorian Sofa.



Points to remember

- ✓ Define one abstract factory interface with one "create" method per product type.
- ✓ Every concrete factory must produce a matching family — never let products from different families mix.
- ✓ Client code should depend only on the abstract factory and abstract product interfaces, never the concrete classes.
- ✓ Adding a brand-new family (e.g. "Scandinavian") means adding one new factory class — no existing code changes.
- ✓ Adding a brand-new product type (e.g. "Table") means touching every existing factory — a known trade-off of this pattern.
- ✓ Keep all products created by the same factory visually/behaviorally consistent with each other.
- ✓ Use it when your code must stay independent of how objects are created, composed, and represented.

Common use cases: cross-platform UI toolkits (light/dark themes), database driver families, game asset/theme packs.

Builder

CoreDesignPattern.Builder

What is it?

Builder constructs a complex object **step by step**. You set only the parts you care about, chaining one call after another, then finish with a single `build()` call that assembles the final object.

Why it was used here

A `Pizza` has one required field (`size`) and several optional toppings (cheese, mushrooms, olives). Instead of a constructor with a long list of booleans, `Pizza.Builder` lets you chain only the toppings you actually want, then call `.build()` . The real `Pizza` constructor is private, so a `Pizza` can only ever be created through its Builder.

```
new Pizza.Builder("Large")
    .cheese()      → returns the Builder (this)
    .mushrooms()   → returns the Builder (this)
    .olives()      → returns the Builder (this)
    .build()       → returns the finished, immutable Pizza
```

Points to remember

- ✓ Make the product's constructor private — it should only ever be built through the Builder.
- ✓ Put required fields in the Builder's own constructor; keep optional ones as chained methods.
- ✓ Every chained "set" method should return `this` , so calls can be linked (a fluent API).
- ✓ The Builder should mirror the exact same fields as the product it's building.
- ✓ Add exactly one `build()` method that constructs and returns the finished object.
- ✓ Aim for the finished product to be immutable once built — no setters afterward.
- ✓ Reach for this pattern when a class has many optional parameters or constructor overloading is getting out of hand.

Common use cases: `StringBuilder`, complex request/config objects, test-data builders, ORM query builders.

Factory Method

CoreDesignPattern.Factory

What is it?

A factory centralizes object creation behind one method, so client code can ask for "a Notification of this type" instead of calling `new EmailNotification()`, `new SMSNotification()`, and so on directly, scattered across the codebase.

Why it was used here

`NotificationFactory.create(NotificationType type)` takes an enum and returns the matching concrete class — Email, SMS, or Push. **Main** never references those concrete classes at all; it only depends on the `Notification` interface and the factory. Adding a new channel later means adding one new class and one new switch case — the client code doesn't change.

```
Notification (interface)
+ send(to, message)
+ getType()
  ^
  |
  +-----+
  |         |         |
EmailNotification SMSNotification PushNotification

NotificationFactory
+ create(NotificationType type)    ← static creation method
  |
  switch (type)
  EMAIL → new EmailNotification()
  SMS   → new SMSNotification()
  PUSH  → new PushNotification()

Client: Notification n = NotificationFactory.create(EMAIL);
       n.send(...)    // never touches "new EmailNotification()" itself
```

Points to remember

- ✓ Client code should depend only on the product interface (`Notification`), never on the concrete classes.
- ✓ All the "which class to instantiate" logic lives in one place — the factory — not scattered across the codebase.
- ✓ Use an enum (or another type-safe key) for the creation parameter instead of a raw string, to catch typos at compile time.
- ✓ Adding a new product type means editing the factory and adding one new class — client code stays untouched.
- ✓ This static version is technically a **Simple Factory** — handy and common, but not quite the classic GoF Factory Method, which uses subclassing so each creator can override what gets built.
- ✓ Keep the factory focused on object creation only — it should build objects, not contain business logic that uses them.
- ✓ Reach for a factory whenever creation involves a decision (a type, a flag, an environment) that shouldn't leak into client code.

Common use cases: notification/messaging channels (as shown), parsers for different file formats, database connection creation, UI element creation.

Singleton

CoreDesignPattern.Singleton

why do we use singleton design pattern?
 It prevents multiple objects from being created,
 saves memory and avoids conflicting states
 managing centralized resources like database connections, logging systems

What is it?

Singleton makes sure a class has **exactly one instance** for the whole application, and gives everyone one shared, global way to reach it.

Why it was used here

`Logger` uses plain lazy initialization: the instance is created the first time `getInstance()` is called, and reused after that — simple, but not safe if two threads call it at the same instant. `ThreadSafeSingletonLogger` fixes that with **double-checked locking**: it checks `instance == null` twice, only synchronizes the block where creation actually happens (not the whole method), and marks the field `volatile` so no thread ever sees a half-built object. Both hide their constructor as `private`, so a `new Logger()` from outside the class is impossible.

```

Logger                                ThreadSafeSingletonLogger
- static instance: Logger              - static volatile instance
- private Logger()                    - private constructor
+ static getInstance():                + static getInstance():
    if (instance == null)              if (instance == null) {
        instance = new Logger();        synchronized (class) {
    return instance;                    if (instance == null)
+ log(message)                          instance = new ...();
                                      }
                                      }
                                      return instance;
                                      + log(message)

logger1 = getInstance(); logger2 = getInstance();
logger1 == logger2 → true (same object – confirmed via hashCode())

```

Points to remember

- ✓ Constructor should be private.
- ✓ Create a private static instance variable.
- ✓ Provide a public static `getInstance()` method.
- ✓ Return the same object every time.
- ✓ Prevent object creation using `new` from outside the class.
- ✓ Use lazy initialization if object creation can be deferred.
- ✓ Make it thread-safe in multithreaded applications.
- ✓ Use Double-Checked Locking (DCL) for efficient thread safety.
- ✓ Mark the instance as `volatile` when using DCL.
- ✓ Synchronize only the object-creation block, not the entire method (for better performance).
- ✓ Verify using `hashCode()` or `==` that only one instance exists.

Common use cases: Logger, Configuration Manager, Cache Manager, Thread Pool Manager, Spring Singleton Beans.

Adapter

CoreDesignPattern.Adapter

What is it?

Adapter lets two incompatible interfaces work together. It's a **translator** that sits in between code that expects one method signature and a class that offers a completely different one.

Why it was used here

`CheckoutService` only knows how to call `PaymentProcessor.processPayment()`. But the real UPI SDK (`UPIGateway`) has a different method entirely — `makePayment(upiId, amountInPaise)` — which `CheckoutService` doesn't understand, and which you can't rewrite since it's third-party. `UPIPaymentAdapter` implements `PaymentProcessor`, wraps a `UPIGateway` inside it, and translates the call: builds the UPI ID, converts rupees to paise, then delegates to the real gateway.

```
CheckoutService
  |
  | expects
  v
PaymentProcessor (Target interface)
  ^
  | implements
  |
UPIPaymentAdapter ← the translator
  |
  | wraps (composition, not inheritance)
  v
UPIGateway (Adaptee – third-party, left untouched)
```

Points to remember

- ✓ Identify the Target interface — the one your client code already expects.
- ✓ Identify the Adaptee — the existing class you can't or don't want to change.
- ✓ The Adapter implements the Target interface.
- ✓ The Adapter holds the Adaptee through composition ("has-a") — never through inheritance.
- ✓ All the messy translation (renaming, unit conversion, param reshuffling) happens inside the Adapter, and nowhere else.
- ✓ The client should never import or reference the Adaptee directly — only the Target.
- ✓ The original Adaptee code stays completely untouched (Open/Closed Principle).

Common use cases: third-party SDKs, legacy code integration, payment gateways, converting between data formats.

Decorator

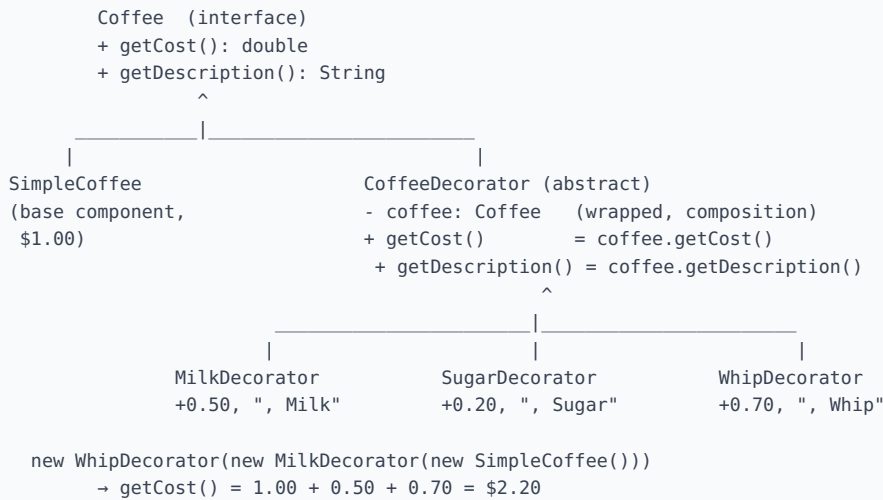
CoreDesignPattern.Decorator — coffee & pizza

What is it?

Decorator lets you add new behavior to an object by **wrapping** it in another object that shares the same interface, instead of creating a new subclass for every possible combination of extra features.

Why it was used here

A **SimpleCoffee** costs \$1.00. Wrapping it in **MilkDecorator** adds \$0.50 and ", Milk" to the description; wrapping that in **SugarDecorator** adds \$0.20 more; wrapping that in **WhipDecorator** adds \$0.70 more. Each decorator calls the coffee it wraps and adds its own bit on top, so any combination — even repeats like double milk — is just another layer of wrapping, with no new class needed. The pizza example (**Margherita** → **CheeseDecorator** → **MushroomDecorator**) applies the exact same idea to toppings.



Points to remember

- ✓ The component interface (e.g. **Coffee**) must be implemented by both the base object and the decorator itself, so they're interchangeable.
- ✓ The abstract decorator holds the wrapped component via composition, and its default methods simply delegate to it.
- ✓ Each concrete decorator calls the wrapped object's method first, then adds its own bit on top — it enhances, it doesn't replace.
- ✓ Decorators can be stacked in any order, and even repeated (e.g. wrapping **MilkDecorator** twice for double milk).
- ✓ This avoids a combinatorial explosion of subclasses that plain inheritance would need (MilkSugarCoffee, MilkWhipCoffee, MilkSugarWhipCoffee...).
- ✓ Client code treats a fully-decorated object exactly like the plain component — same interface, so it can be swapped in anywhere (Liskov Substitution Principle).
- ✓ Be intentional about wrapping order if the decorators aren't purely additive — order can change the result.
- ✓ Keep each decorator focused on exactly one added responsibility.

Common use cases: Java I/O streams (wrapping Reader/Stream), UI component styling (scrollable, bordered), order customization like this coffee/pizza example, request/response middleware.

Facade

CoreDesignPattern.Facade

What is it?

Facade puts one simple, easy-to-use interface in front of a complex subsystem made of several moving parts, hiding the coordination detail behind a single entry point.

Why it was used here

Watching a movie actually needs seven steps across four different subsystems — dim the lights, turn on the projector, set its input, turn on the amplifier, enable surround sound, set the volume, then start the DVD. `HomeTheaterFacade` wraps `Projector`, `Amplifier`, `DVDPlayer`, and `Lights`, and does all seven steps in the right order behind one call: `watchMovie("Inception")`. The client in `FacadeDemo` never has to know that order, or that four separate objects are even involved.

```
Client
|
v
HomeTheaterFacade    ← single entry point
+ watchMovie(movie)
+ endMovie()
|
+----> Projector      (subsystem A)
+----> Amplifier      (subsystem B)
+----> DVDPlayer      (subsystem C)
+----> Lights         (subsystem D)
```

Facade coordinates the subsystems; they don't know about each other.
Client can still call a subsystem directly if fine-grained control is needed.

Points to remember

- ✓ A Facade doesn't add new functionality — it simplifies access to functionality that already exists in the subsystem.
- ✓ The Facade coordinates calls across multiple subsystem objects in the right order, so the client doesn't have to.
- ✓ Subsystem classes should stay unaware of the Facade — they don't call back into it, and they don't know about each other.
- ✓ The client isn't forced to go through the Facade — direct subsystem access should still be possible when fine-grained control is genuinely needed.
- ✓ Keep the Facade thin: it should delegate work to subsystems, not contain business logic of its own.
- ✓ Add one Facade method per meaningful client use case (e.g. `watchMovie`, `endMovie`), not one per subsystem method.
- ✓ A Facade lowers coupling between client code and subsystem internals — subsystems can change internally as long as the Facade's own interface stays stable.
- ✓ Don't let the Facade grow into a god object — it should coordinate the subsystems, not absorb their responsibilities.

Common use cases: simplifying complex library/API usage, home automation (as shown), wrapping legacy subsystems, hiding a multi-step setup sequence behind one call.

Proxy

CoreDesignPattern.Proxy

What is it?

Proxy puts a stand-in object in front of the real object. The proxy implements the exact same interface, so the client can't tell it apart from the real thing — but the proxy decides what actually happens before (or instead of) forwarding the call.

Why it was used here

`FileAccessProxy` implements `FileAccess` and wraps a `RealFileAccess`. Before forwarding `deleteFile()`, it checks the caller's role — only `"ADMIN"` gets forwarded to the real object; anyone else is denied, and `RealFileAccess` is never touched. The second example, `ProxyImage`, uses the same idea for lazy loading: it only creates the expensive `RealImage` (which loads from disk) on the *first* call to `display()`, then reuses that cached instance on every call after — a "Cache Hit" instead of a slow reload.

```

FileAccess (Subject interface)
+ deleteFile(fileName)
    ^
   / \
  /   \
RealFileAccess      FileAccessProxy  ← same interface
(does the real work)
- role: String
- realFileAccess: RealFileAccess
+ deleteFile(fileName)
    role == "ADMIN" ?
        yes → delegate to RealFileAccess
        no  → deny, real object untouched
  
```

Points to remember

- ✓ The proxy implements the same interface as the Real Subject, so it's a drop-in replacement — the client can't tell the difference.
- ✓ The proxy holds a reference to the Real Subject (composition), and controls when — or whether — calls actually reach it.
- ✓ A protection proxy checks permissions before delegating, and simply skips calling the real object if access is denied.
- ✓ A virtual proxy defers creating the expensive real object until it's actually needed, then caches it for reuse.
- ✓ Keep the proxy's own logic (checks, caching) separate from the real object's business logic — the real object shouldn't need to know a proxy exists.
- ✓ Don't confuse Proxy with Decorator: Decorator adds new behavior/responsibilities, while Proxy controls access to behavior that already exists.
- ✓ Other common proxy jobs beyond these two: logging, remote calls, rate limiting.

Common use cases: access control (as shown), lazy-loading of expensive resources like images or DB connections (as shown), remote proxies (RPC stubs), caching layers.

Chain of Responsibility

CoreDesignPattern.Chain_of_Responsibility

What is it?

Chain of Responsibility passes a request along a line of handlers. Each handler independently decides: **handle it myself**, or **pass it to the next one in line**.

Why it was used here

A leave request first reaches **TeamLead**. If it's 2 days or fewer, they approve it directly. If it's longer, they forward it to **Manager**. If it's still too long, **Manager** forwards it to **Director**, who approves anything. The requester never needs to know who ultimately approves the request — they simply hand it to the first link in the chain.



Points to remember

- ✓ Give every handler a reference to the "next" handler in the chain.
- ✓ Each handler decides for itself: handle the request, or forward it — never silently do both.
- ✓ The chain is assembled by the client (or a config step) — individual handlers only know their immediate neighbor.
- ✓ Always check `next != null` before forwarding, or you'll hit a null pointer at the end of the chain.
- ✓ Keep each handler's condition simple and focused on one responsibility.
- ✓ The order in which you link handlers matters — build the chain deliberately.
- ✓ Decide up front what happens if no handler in the chain accepts the request — don't let it silently disappear.

Common use cases: approval workflows, HTTP middleware/filters, logging pipelines, exception-handling chains.

Observer

CoreDesignPattern.Observer

What is it?

Observer lets one object (the Subject) automatically notify a list of other objects (Observers) whenever something changes, without the Subject needing to know anything about them beyond a shared interface.

Why it was used here

`YouTubeChannel` (Concrete Subject) keeps a list of `Subscriber` s. When `uploadVideo()` is called, it updates `latestVideo` and calls `notifySubscribers()` , which loops through every subscriber and calls `update()` on each. Every `User` (Concrete Observer) just prints what it received — the channel never needs to know how many users are subscribed, or what they do with the notification, only that they implement `Subscriber` .

```

Channel  (Subject interface)
+ subscribe(Subscriber)
+ unsubscribe(Subscriber)
+ notifySubscribers()
    ^
    |
YouTubeChannel  (Concrete Subject)
- subscribers: List<Subscriber>
- latestVideo: String
+ uploadVideo(title)
    latestVideo = title;
    notifySubscribers(); → loops subscribers, calls each User.update(...)

Subscriber  (Observer interface)
+ update(newVideoUploaded)
    ^
    |
User  (Concrete Observer)
+ update(...) → prints it

```

Points to remember

- ✓ The Subject holds a collection of observers and exposes subscribe/unsubscribe methods to manage it.
- ✓ The Subject notifies observers only through the shared Observer interface — it never depends on concrete observer classes.
- ✓ Observers register themselves with the subject; the subject doesn't go looking for them.
- ✓ Keep the notification payload minimal, or pass the subject itself, so observers can pull whatever extra detail they need.
- ✓ Always support unsubscribe, or you'll accumulate observers that should be gone but are still referenced (a memory leak).
- ✓ Watch out for observers that modify the subscriber list during notification — that's a classic concurrent-modification bug.
- ✓ Observers shouldn't assume a particular notification order, and shouldn't do slow work inside `update()` — that blocks the whole notification loop.

Common use cases: UI event listeners, pub/sub systems, Model-View updates (MVC), live feeds and notifications like this example.

Strategy

CoreDesignPattern.Strategy

What is it?

Strategy lets you swap out an algorithm or behavior at runtime by wrapping each variant behind a common interface, instead of hard-coding one implementation or writing a long if/else chain to pick between them.

Why it was used here

`PaymentProcessor` (Context) holds a `PaymentStrategy` and calls `.pay()` without caring which concrete strategy it is. `CheckoutFlow` starts with `CreditCardStrategy`, then swaps to `PayPalStrategy`, then `CryptoStrategy` via `setStrategy()` — all three checkouts go through the exact same `processor.checkout(amount)` call.

```
PaymentStrategy (interface)
+ pay(amount): boolean
+ methodName(): String
    ^
   / | \
  /  |  \
CreditCardStrategy PayPalStrategy CryptoStrategy

PaymentProcessor (Context)
- strategy: PaymentStrategy
+ setStrategy(strategy)
+ checkout(amount) → strategy.pay(amount)

processor = new PaymentProcessor(new CreditCardStrategy(...));
processor.checkout(99.99); // via Credit Card
processor.setStrategy(new PayPalStrategy(...));
processor.checkout(49.50); // via PayPal
processor.setStrategy(new CryptoStrategy(...));
processor.checkout(200.00); // via Crypto
```

Points to remember

- ✓ Define one strategy interface that every interchangeable algorithm/behavior implements.
- ✓ The Context class holds a reference to a strategy and delegates the actual work to it — it never implements the algorithm itself.
- ✓ The strategy can be swapped at runtime through a setter, without changing the Context's own code.
- ✓ Client code decides which concrete strategy to use — the Context stays generic.
- ✓ Adding a new strategy means adding one new class — no existing strategy or the Context needs to change (Open/Closed Principle).
- ✓ Keep each concrete strategy focused on exactly one way of doing the job — no branching for "modes" inside a single strategy class.
- ✓ Reach for Strategy once the number of variants grows, they need to be swapped at runtime, or each one carries its own state (like the email, card number, and wallet address here).

Common use cases: payment methods (as shown), sorting/comparison strategies, compression algorithms, validation rules.